Reliable, Scalable and Maintainable Applications

Data-Intensive Systems

- Compute-intensive vs. data-intensive
- Amount of data
- Complexity of data
- Speed at which it is changing

Data-Intensive Applications

- A data-intensive application is typically built from standard building blocks.
- Many applications need to:
  - Store data – *databases*
  - Remember the results of expensive operations – *caches*
  - Allow users to search data by keyword or filter in various ways – *search indexes*
  - Periodically crunch a large amount of accumulated data – *batch processing*
  - Send a message to another process, to be handled asynchronously – *stream processing*

Data Systems

- Data systems are such a successful abstraction.
  - We do not write a new data storage engine from scratch when building an application.
  - Databases are perfectly good tool.
- But,
  - Many database systems with different characteristics
  - Various approaches to caching, several ways of building search indexes, and so on

Data Systems

- When building an application, we still need to figure out
  - which tools and
  - which approaches are the most appropriate for the task at hand.

Three Main Concerns for Data Systems

- Reliability
- Scalability
- Maintainability

Reliability Features of Software

- Meeting User Expectations
  - The application successfully performs the functions that the user expects.
- Error Tolerance
  - It can tolerate user mistakes or unexpected usage of the software.
- Adequate Performance
  - The application provides good enough performance for the required use case, under the expected load and data volume.
- Unauthorized Access Prevention
  - The system prevents any unauthorized access and abuse.

Reliability Features of Software

All those things together mean “working correctly”

- Meeting User Expectations
  - The application successfully performs the functions that the user expects.
- Error Tolerance
  - It can tolerate user mistakes or unexpected usage of the software.
- Adequate Performance
  - The application provides good enough performance for the required use case, under the expected load and data volume.
- Unauthorized Access Prevention
  - The system prevents any unauthorized access and abuse.

Reliability

- “Continuing to work correctly, even when things go wrong.”
- The things go wrong – *faults*
- Systems that anticipate faults and can cope with them – *fault-tolerant* or *resilient*
- Faults can be cured: hardware faults, software errors, human errors

Hardware Faults

- Common Hardware Issues:
  - Hard disk crashes
  - Faulty RAM
  - Power grid blackouts
- Frequency of Failures:
  - These issues occur regularly, especially with numerous machines in use.
- First Response:
  - Add Redundancy: Implement redundancy in individual hardware components.

Hardware Faults

- Enhancing Machine Resilience:
  - Single machines are made more resilient through redundant hardware.
- Increasing Demands:
  - As data volumes and computing demands have risen, more applications require larger numbers of machines.
- Consequences:
  - This increase leads to a higher rate of hardware faults.
- Evolving Strategies:
  - Software Fault-Tolerance Techniques: Shift towards utilizing software fault-tolerance methods either in addition to or instead of hardware redundancy.

Software Errors

- Bugs are typically systematic and hard to deal with.
- No quick solution – lots of small things can help:
  - Carefully thinking about assumptions and interactions in the system
  - Thorough testing
  - Process isolation
  - Allowing processes to crash and restart
  - Measuring, monitoring, and analyzing system behavior in production

Human Errors

- How do we make our systems reliable, in spite of unreliable humans?
  - Minimize opportunities for human error
  - Decouple places where people make the most mistakes from places where mistakes can cause failures
  - Test thoroughly at all levels, from unit tests to whole-system integration tests and manual tests
  - Allow quickly and easy recovery from human errors
  - Set up detailed and clear monitoring, such as performance metrics and error rates

Scalability

- If a system is working reliably today, does it mean it will necessarily work reliable in the future?
  - 1 million users to 10 million users?
- Scalability – a system's ability to cope with increased load

Describing Load

- Use metrics called *load parameters*
  - Ex*:* Twitter data in November 2012
    - Post tweet: 4.6k requests/sec on average, 12k requests/sec at peak
    - Home timeline: A user can view tweets posted by the people they follow (300k requests/sec).
    - Twitter has different designs to deal with updating home timelines

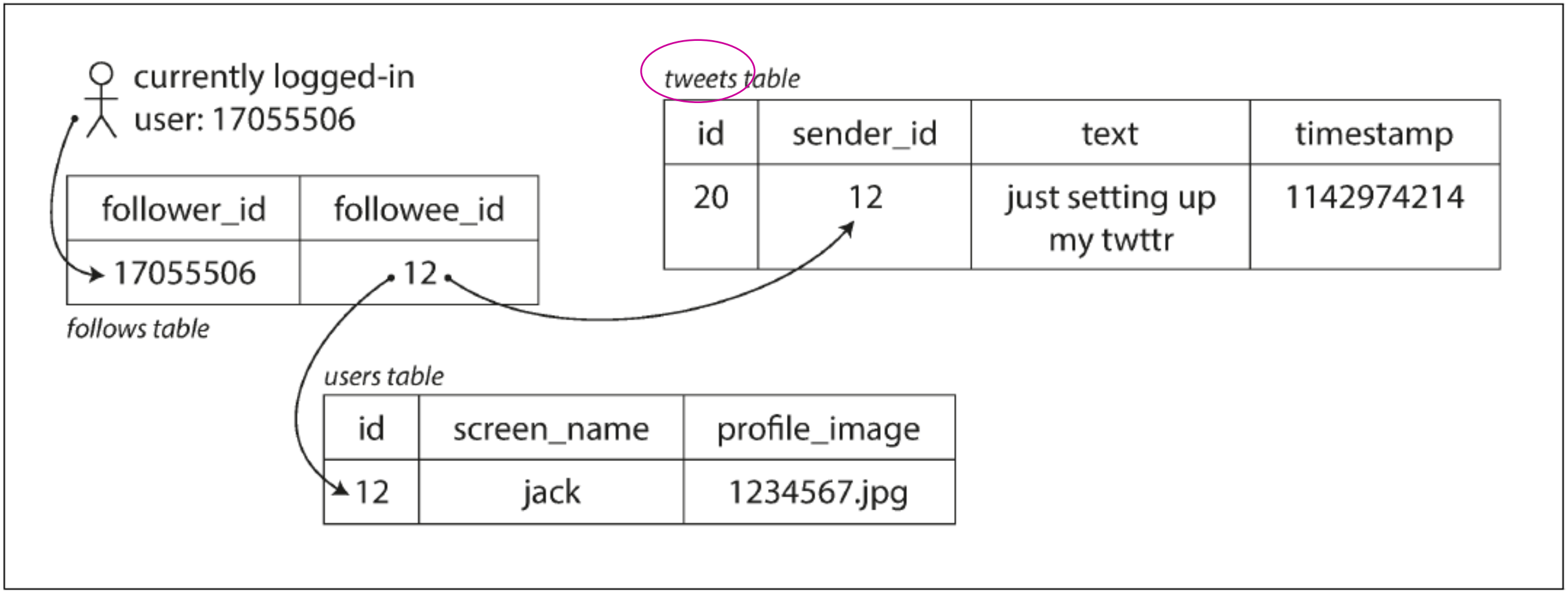


Figure 1-2. Simple relational schema for implementing a Twitter home timeline.

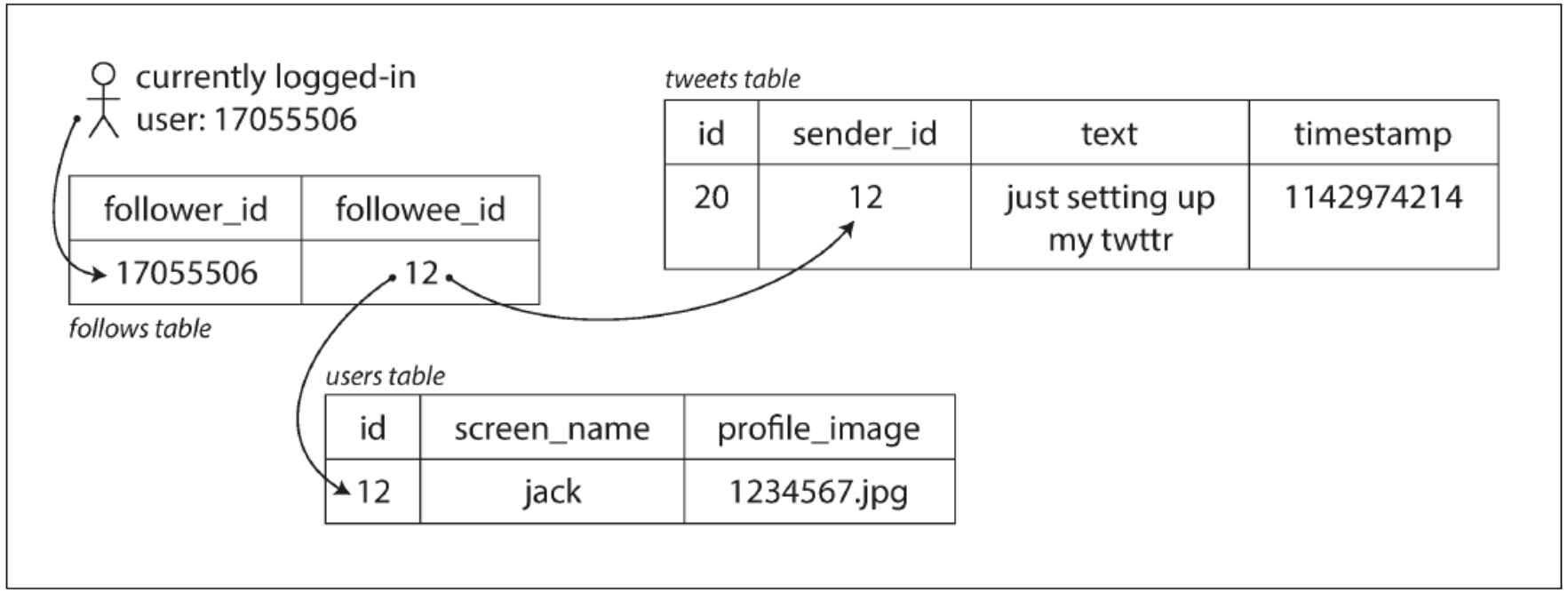


Figure 1-2. Simple relational schema for implementing a Twitter home timeline.

```
SELECT tweets.*, users.* FROM tweets
  JOIN users   ON tweets.sender_id    = users.id
  JOIN follows ON follows.followee_id = users.id
  WHERE follows.follower_id = current_user
```

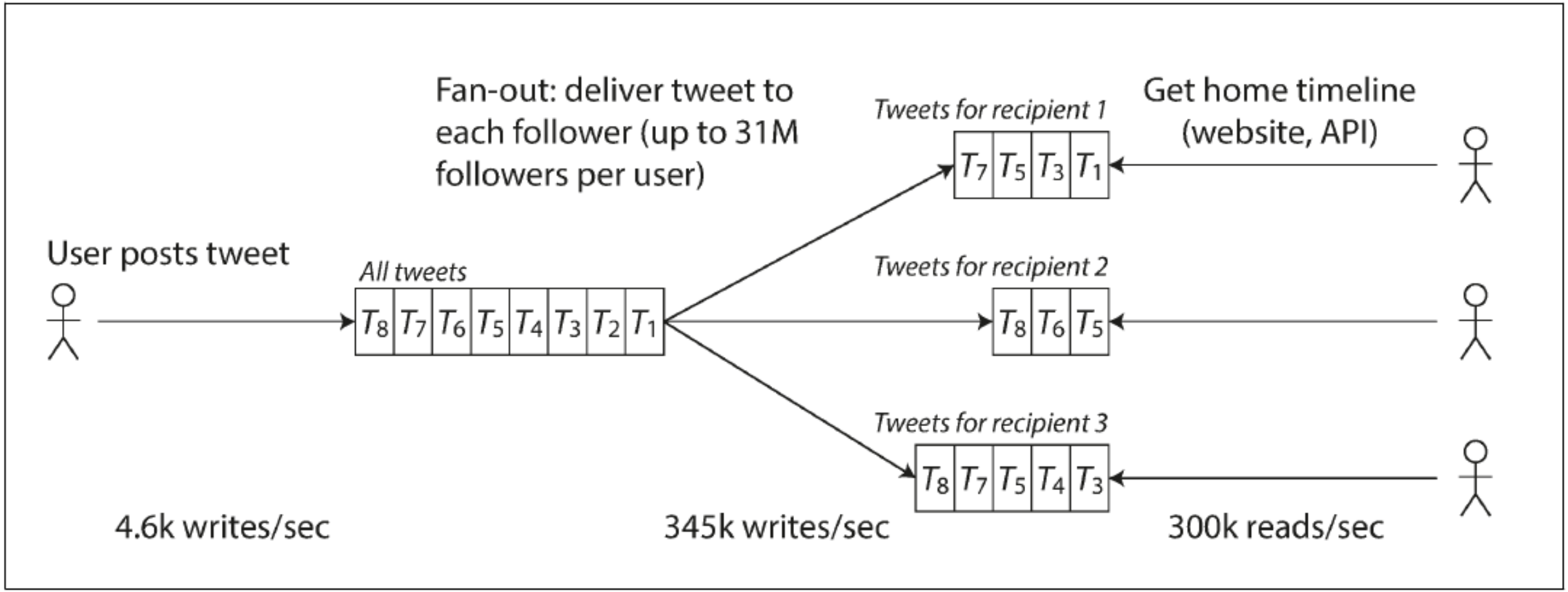


Figure 1-3. Twitter's data pipeline for delivering tweets to followers, with load parameters as of November 2012 [16]. Maintain a cache for each user's home timeline

Describing Load

- The first version of Twitter used approach 1
- But the systems struggled to keep up with the load of home timeline queries, so the company switched to approach 2
  - The average rate of published tweets is almost two orders of magnitude lower than the rate of home timeline reads
  - So, in this case it's preferable to do more work at write time and less at read time
  - Downside of approach 2?

Describing Load

- Downside of approach 2?
  - Posting a tweet requires a lot of extra work
  - Some users have over 30 million followers – means that a single tweet may result in over 30 million writes to home timelines!

Describing Load

- In the example of Twitter, a key load parameter for discussing scalability:
  - The distribution of followers per user (maybe weighted by how often those users tweet)
- Your application may have very different characteristics, but you can apply similar principles to reasoning about its load

Investigate What Happens When the Load Increases

- When you increase a load parameter and keep the system resources unchanged
  - How is the performance of your system affected?
- When you increase a load parameter
  - How much do you need to increase the resources to keep performance unchanged?

- Both questions require performance numbers

Describing Performance

- Throughput
  - The number of records we can process per second, or
  - The total time it takes to run a job on a dataset of a certain size
- Response time
  - The time between a client sending a request and receiving a response
  - Average response time

Describing Performance

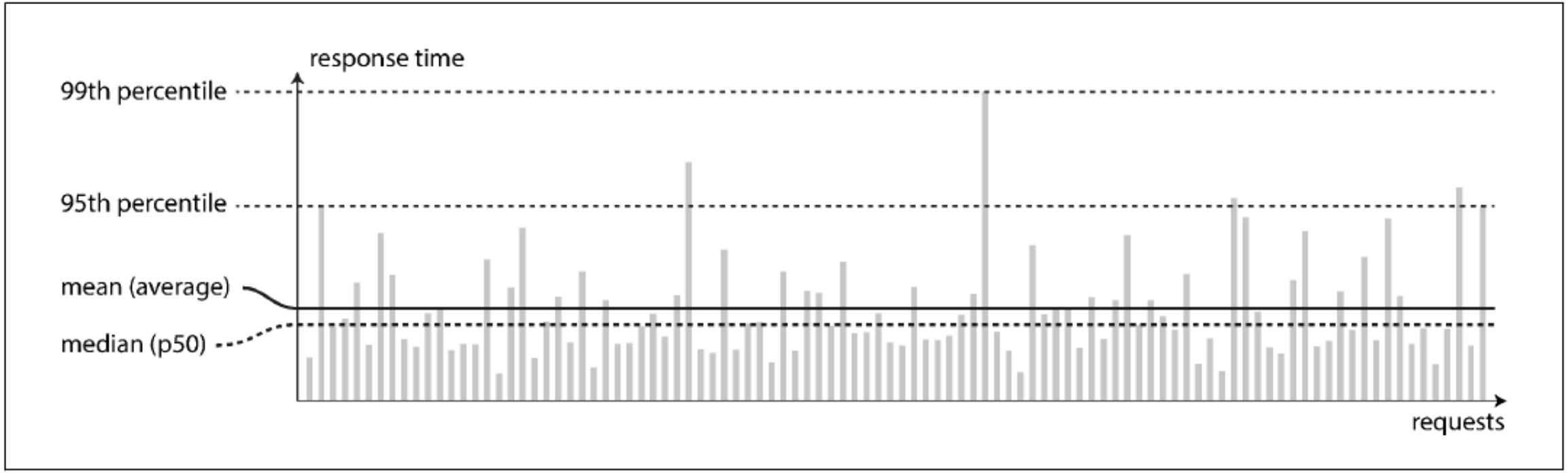
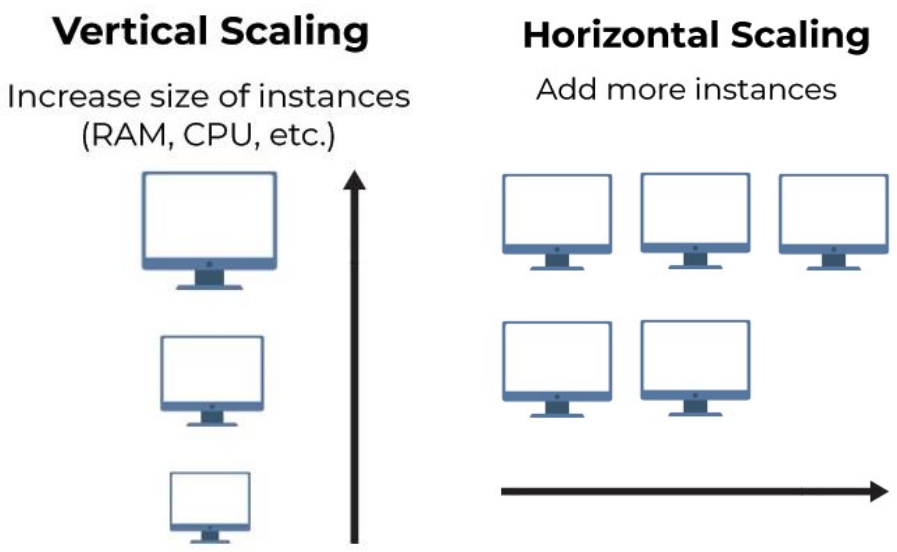


Figure 1-4. Illustrating mean and percentiles: response times for a sample of 100 requests to a service.

Approaches for Coping with Load

- How do we maintain good performance even when our load parameters increase by amount?
  - Scaling up
    - Vertical scaling
    - Moving to a more powerful machine
  - Scaling out
    - Horizontal scaling
    - Distributing the load across multiple smaller machines



https://www.spiceworks.com/tech/cloud/articles/horizontal-vs-vertical-cloud-scaling/

Maintainability

- Most of the cost of software is not in its initial development, but in its ongoing maintenance
  - Fixing bugs
  - Keeping its systems operational
  - Adapting it to new platforms
  - Modifying it for new use cases
  - Adding new features
  - …

Maintainability

- We should design software in such a way that it will hopefully minimize pain during maintenance
- Design principles for software systems:
  - Operability: Making life easy for operations - making routine tasks easy
    - Make it easy for operations teams to keep the system running smoothly
  - Simplicity: Managing complexity
    - Make it easy for new engineers to understand the system, by removing as much as complexity as possible from the system
  - Evolvability: Making change easy
    - Make it easy for engineers to make changes to the system in the future

Three Main Concerns for Data Systems

Reliability

- Tolerating hardware & software faults
- Human error

Scalability

- Measuring load & performance
- Latency percentiles, throughput

Maintainability

- Operability, simplicity & evolvability

Conclusion

- There is unfortunately no easy fix for making applications reliable, scalable, or maintainable.
- There are certain patterns and techniques that keep reappearing in different kinds of applications.
- We will look at some examples of data systems and analyze how they work toward those goals.

References

- M. Kleppman. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, Inc., 2017.